HPC Assignment 3: Lennard-Jones Simulation

Žane Bučan, Gregor Bučar

I. Introduction

Molecular dynamics simulations are a powerful computational method used to study the behavior of systems composed of many interacting particles. By modeling the forces between particles and numerically integrating their motion over time, these simulations help us understand how complex physical phenomena emerge from simple interactions.

II. Problem description

In this assignment, the goal is to implement an efficient parallel simulation of a two-dimensional system of particles interacting via the Lennard-Jones potential. The main challenge lies in optimizing the computation of pairwise forces and leveraging GPU parallelism (CUDA) to handle large numbers of particles while maintaining accuracy and performance.

III. Optimizations

A. GPU

First improvement was to use two pointers for the particle arrays when CREATE_GIF is ON. One on the device (d_particles) and one on the host (h_particles_pinned). Why we used another pointer on the host? The difference between h_particles_pinned and normal particles is that h_particles_pinned use **cudaHostAlloc()** which pins the memory making it so that the GPU can directly access it. With this, the code avoids explicit **cudaMemcpy()** calls inside the main simulation loop, because both host and device can access this memory. Of course the algorithm scraps this idea when GENERATE_GIF is OFF, because then, the CPU does not need data after every step to generate a frame in the gif, so the GPU can have the particles all for it self, therefore there is no need of creating a pinned memory, but just copy every particle straight to the GPU memory for faster access.

Second improvement was to add CUDA kernels to the simulation. We added them to these steps:

- Computing of kinetic energy
- Computing of forces and potential energy
- Leapfrog step

Leapfrog was split into three parts, one updating the particle positions and velocity for 0.5 a step, second computing forces and another finishing the update of the velocity. With this, we improve the accuracy of the computation.

We also added a sum_array_kernel which uses the best practices to sum up the array in parallel. It uses shared memory sun_cache so that the threads can access sub sums even faster. It was added for a faster energy computation for the whole particle system.

Another improvement is that we did not create a separate GPU function for the warp positions for the GPU, but we included it straight into the leapfrog step.

The last change that we can not ignore is the use of 3rd Newtons law and acknowledging that when a particle uses its force on another, that particle reacts with an equal and opposite force. With that we reduced the nested for loop in the compute_forces_kernel() from $O(N^2)$ to $O\left(\frac{N(N-1)}{2}\right)$ and only need one force computation for two particles. The downside of this is that we need to ensure each thread reads the correct value if two threads calculate values, if two or more threads compute forces to pairs which share the same particle. We achieved that by using **atomicAdd()**, which is not super ideal, because now threads could wait for each other. So with our approach there is a plus and a minus to the algorithm.

B. CPU

We did not complicate with this improvement and simply added OpenMP to the proper loops.

We also included Newtons 3rd law to the CPU computation and with that introduced atomic writhing, which again could be a minus and a plus, because the iterations of the nested loops decrease while the CPU threads could wait for one another.

IV. Experiment

Each improvement was tested on a 5000 step simulation with 1000, 2000, 4000 and 8000 particles. The CPU implementation was tested on 16, 32, 64 and 128 cores while GPU was tested with block sizes of 32, 64, 128, 256, 512 and 1024 threads per block. We ran the algorithm 5 times with each setup and averaged the time.

V. Results

These are the results of the benchmark

Particles	CPU Cfg	CPU Time (s)	CPU Speedup
1000	64	1.571	11.02x
2000	128	3.331	19.57x
4000	128	10.461	23.61x
8000	128	33.391	28.90x

Table I: Best CPU configuration performance and speedup relative to CPU baseline (1 core).

Particles	GPU Cfg	GPU Time (s)	GPU Speedup (Baseline)
1000	128	5.090	3.40x
2000	128	6.073	10.74x
4000	128	9.829	25.13x
8000	128	17.755	54.35x

Table II: Best GPU configuration performance and speedup relative to CPU baseline.

Particles	GPU Speedup vs CPU	Interpretation
1000	0.31x	GPU slower than CPU
2000	0.55x	GPU slower than CPU
4000	1.06x	GPU slightly faster
8000	1.88x	GPU significantly faster

Table III: Relative performance of GPU compared to best CPU configuration. Values > 1 indicate GPU advantage.

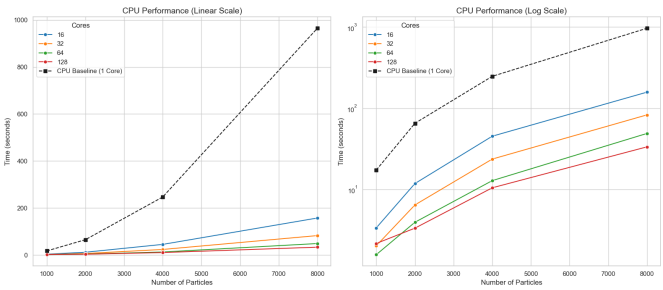

Figure 1: Optimized CPU performance vs baseline 1 core CPU

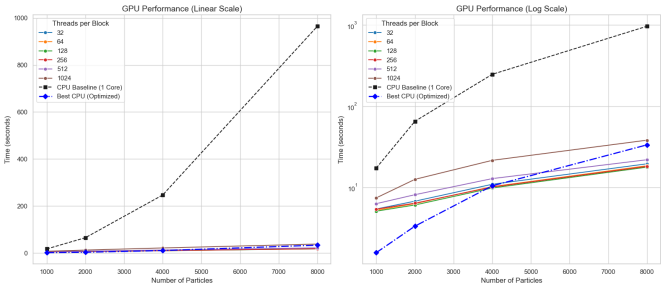

Figure 2: Optimized GPU performance vs baseline 1 core CPU and optimized CPU

Interesting result is that the optimized CPU was actually faster with smaller number of particles, but when you increase the number of them, the GPU starts to dominate.

Another benefit of using the GPU is that with larger number of particles, the execution time does not increase as much as the CPU execution times.

VI. Conclusion

In this work, we implemented and optimized a two-dimensional Lennard-Jones molecular dynamics simulation using both CPU (OpenMP) and GPU (CUDA) parallelization. Through a series of optimizations, including memory pinning, kernel fusion, parallel reductions, and the application of Newton's Third Law, we significantly reduced computational overhead in the force calculation stage while maintaining physical correctness.

For smaller systems, the optimized CPU version remains competitive or even faster. However, as the number of particles increases, the GPU consistently outperforms the CPU, achieving substantial speedups and confirming its advantage for large scale particle simulations.

Overall, the results highlight the importance of matching algorithmic structure with appropriate hardware. While CPU optimizations improve performance in low to medium regimes, GPU acceleration becomes essential for efficiently handling large-scale molecular dynamics simulations.